Hello, World

> A small phrase, spoken through ninety-nine different machines.

Pretext: Before the First Voice

There is a small misunderstanding at the entrance to this book. The reader may assume that the phrase `Hello, World!` is an announcement: a machine has awakened, a person has instructed it to speak, a screen has answered with obedient light. That is the useful account, and usefulness is often the coat reality wears in public. But the phrase may also be a question wearing a greeting as a disguise. Who is speaking when the letters appear? The person who wrote them has already stepped away. The machine has no mouth, no morning, no private recollection of rain against a window. Yet something has crossed a threshold. A silence has accepted a pattern. The pattern has been made visible. In the ordinary world this is called output. In the world just beneath it, where the floorboards remember every footstep and the wires have long conversations after everyone has gone home, it is called an arrival.

You are not required to believe in the hidden world for it to affect the visible one. A key does not need a philosophy of locks to turn. A sentence does not need to understand the person reading it in order to lodge somewhere behind the eyes. Still, it is worth noticing that every program begins by asking matter to participate in imagination. A sequence of marks, arranged with enough care, becomes a request. The request passes through layers of decisions made by absent people: people who designed instructions, named operations, defined characters, built languages, decided that one tiny electrical condition should mean this and another should mean that. The request is translated so many times that it ought to arrive exhausted. Instead, it arrives fresh, bright, and unembarrassed. The screen says hello as though no one had ever said it before.

Perhaps that is why the phrase survives. It is not clever. It does not prove a theorem, balance an account, guide a vehicle, predict a storm, or keep a distant system alive through the night. It is smaller than usefulness and therefore difficult to replace. It is a ceremonial first stone. It tells the beginner that the invisible country can be entered. It tells the expert that the country is still invisible, no matter how many roads have been drawn across it. The sentence is a handshake between the one who wonders and the thing that will not explain itself. It is a

lamp switched on in an unoccupied room, a way of discovering that the room has been there all along.

The languages ahead do not agree about much. Some arrive with a few plain words and an open door. Some require a house, a procedure, a declaration, an import, a contract, a module, a proof, an offering to the ancient gods of punctuation. Some want the world to be a string. Some want it to be an effect. Some regard it as a value, an object, a message, a signal, a record, a process, a statement at the beginning of time. They do not merely encode the same thought differently. They reveal different suspicions about thought itself. Is expression dangerous unless checked? Is freedom more useful when fenced? Is the machine a servant, a partner, a stage, a wilderness, a ledger, an instrument, a city whose lights are all controlled from somewhere else? Each answer leaves fingerprints on the smallest possible greeting.

Read the code slowly, if slowness is available to you. Do not read it as a museum label. Read it as a series of local weather reports from distant climates of mind. Notice how one language points outward, another downward, another toward an enclosing structure. Notice the bracket, the colon, the indentation, the word that names the beginning. Notice that a semicolon can feel final in one place and decorative in another. Notice that a newline sometimes has to be invited, sometimes is granted automatically, sometimes waits outside the door until somebody remembers it. These are tiny choices, but a civilization can hide inside a tiny choice. A civilization is, after all, only a pattern repeated until people mistake it for the shape of the world.

There is a temptation to ask which language is best. The temptation is understandable; the mind enjoys a shelf with labels, a mountain with a summit, a contest with a bell at the end. But this book has no winner because the phrase it contains has no rivalry. Each entry is a different vessel carrying the same impossible cargo: a declaration that there is a world beyond the code, and that the code has noticed it. One vessel is ornate, another severe, another playful, another almost forgotten. A few are still in daily use; others survive more as echoes, old rooms with their lights left on. The greeting does not care. It takes the shape it is given and steps into the air.

If the text begins to feel repetitive, trust the feeling. Repetition is not the opposite of revelation. A clock repeats itself while dividing darkness from daylight. A corridor repeats itself while leading somewhere new. The phrase returns because returning is one of the ways a thing proves it has changed. You will see `Hello, World!` again and again, but you will not meet the same hello twice. The context shifts. The surrounding machinery shifts. You shift. Even the reader who believes they are merely browsing has already become the recipient of a hundred small invocations. The book is not asking you to learn every language. It is asking

you to notice the moment before learning becomes possession, when the unknown is still allowed to be strange.

So begin here, before the first language has announced itself. Consider the blank space above the first heading. Consider that it is not empty, only unaddressed. Consider that every technical thing you have ever trusted was once a line someone wrote without knowing every consequence it would have. Then let the first voice speak. It will greet the world in its own accent. The world, being older than any machine and younger than every new idea, will answer by continuing to exist.

01. Ada

```
with Ada.Text_IO; use Ada.Text_IO;

procedure Hello_World is
begin
    Put_Line ("Hello, World!");
end Hello_World;
```

Ada enters wearing a perfectly tailored spacesuit, carrying a checklist that somehow feels like a hymn. It says hello with the calm of a system that has considered the cost of every loose bolt. The greeting passes through named rooms and deliberate doors. Nothing is accidental; even the silence has been assigned a sensible type. Far above the ordinary screen, a satellite receives the phrase and keeps orbiting. Reliability, it turns out, can be a form of devotion.

02. ALGOL 68

```
BEGIN
  print(("Hello, World!", newline))
END
```

ALGOL 68 emerges from a clean, strange corridor in the history of thought. Its punctuation is a map drawn by people who believed a program might be both an engine and a proof. The world is printed between formal walls. It does not complain; it has always enjoyed a little ceremony when entering a room. Some languages are old because they have faded. This one is old because an idea can keep casting a long shadow after the lamp is gone.

03. APL

```
⎕←'Hello, World!'
```

APL writes the greeting with symbols that seem to have escaped from the margin of a mathematics textbook and learned how to glow. One glyph opens its hand, one string answers, and the console receives a message that looks less typed than summoned from a tidy storm.

04. Assembly (x86-64)

```
global _start

section .data
    message db "Hello, World!", 10
    message_length equ $ - message

section .text
_start:
    mov rax, 1
    mov rdi, 1
    mov rsi, message
    mov rdx, message_length
    syscall
    mov rax, 60
    xor rdi, rdi
    syscall
```

Assembly does not say hello. It moves the letters into the right registers and opens the correct door in the kernel, which is its own austere form of tenderness. The greeting crosses the threshold one byte at a time, wearing no costume except the exact number of bytes it needs to survive.

05. ATS

```
implement main0 () = println! ("Hello, World!")
```

ATS arrives carrying proofs in one pocket and systems programming in the other, refusing to pretend that safety and speed must live in separate countries. Its hello is brief, but behind it stands a whole tribunal of types asking whether reality has been checked carefully enough.



A first spark in the dark.

06. AWK

```
BEGIN { print "Hello, World!" }
```

AWK finds the world in a line of text and immediately asks what field it belongs to. It has the alert, unglamorous magic of a person who can repair a radio with a paperclip. The program is a single match struck against a data stream. For a moment, the whole river flashes with a readable name. Then the records keep flowing. AWK turns back to the current, sleeves rolled up, delighted by the next small pattern.

07. Bash

```
echo "Hello, World!"
```

Bash speaks from the border between thought and command. It knows the names of files, the moods of permissions, and the midnight language of machines doing their chores. `echo` is a voice in the workshop. No abstraction need stand between the speaker and the air; the phrase is sent out with a tap of the hammer. The shell remembers that computers are places as well as ideas. Behind every prompt is a hallway, a drawer, a door that can be opened.

08. BCPL

```
GET "LIBHDR"

LET START () BE
        WRITEF("Hello, World!*N")
```

BCPL sends the greeting from a workshop near the beginning of modern systems programming, where the tools are blunt, useful, and still warm from having shaped several descendants. The asterisk in its newline feels like an old signal lamp, blinking once across a room full of history.

09. BQN

```
•Out "Hello, World!"
```

BQN writes as if the keyboard were a cabinet of rare mathematical birds. Symbols gather, tilt their heads, and perform operations that feel like weather inside a mirror. Here, a greeting is not merely said. It is placed in the care of an unfamiliar glyph and sent out through a bright, compact portal. The strange alphabet does not close the door. It reminds us that thought has more costumes than the usual hallway permits.

10. C

```
#include <stdio.h>

int main(void) {
    printf("Hello, World!\n");
    return 0;
}
```

C is an old brass key turning in a planetary lock. It asks for a doorway, a breath of memory, a clear exit; then it speaks with the economy of a lighthouse that has seen too many ships. C remembers when computers were rooms with weather systems. It speaks close to the floorboards, where wires have names and a mistake can smell faintly of hot metal. `main` is not a metaphor in this language. It is the door marked MAIN, painted in block letters at the end of a concrete corridor, and the program enters without looking back. The include directive is an invocation. A header rises from the archive like a useful ghost, bringing with it

the ancient ability to let letters escape into the night. There is no illusion that the world is soft. A byte is a byte. A pointer is a finger extended into the dark, and it matters enormously what it touches. Yet this severity has its music. The program speaks because somebody arranged the right stones in a river and trusted the water to recognize a crossing. `return 0` is a small bow at the end of a dangerous dance. Nothing caught fire. The function has come back from the edge carrying a clean, empty flag. The greeting has weight here. It was lifted from memory, marched through instructions, and set down carefully before us as if it were a crate labeled FRAGILE: CONTAINS DAWN.



Signals crossing a quiet sky.

11. C#

```
Console.WriteLine("Hello, World!");
```

C# places the phrase on a silver tray. It has arrived from a clean-lined city where abstractions wear tailored coats and every corridor knows the difference between public and private. The line falls into the console with a clear chime. Somewhere, a garbage collector quietly sweeps the footprints from the corridor, leaving the greeting immaculate. C# believes the universe can be made legible by arranging its drawers well. It does not deny the storm outside; it merely gives the storm a namespace and a reliable set of exceptions. The capital letters stand upright like municipal buildings. `Console`, `WriteLine`: each word knows its office, its hours, and the elegant ceremony required to make a sentence visible. There is a particular peace in managed memory. The program may create a thought, use it brightly, and leave the sweeping to an unseen custodian with a silver broom. C# takes the loose, warm animal of intention and gives it a badge. It says, you may enter the city, but please observe the lanes, the signals, the beautifully painted lines. The console is an old black stage beneath all the glass. On it, the phrase appears without scenery, which is why it feels almost honest. Perhaps order is not the enemy of wonder. Perhaps it is the table laid before wonder arrives, so that wonder does not have to eat standing up. The line concludes, immaculate and

audible. For one moment, the enormous managed world is a bell held still in the hand before it rings again.

12. C++

```
#include <iostream>

int main() {
    std::cout << "Hello, World!\n";
}
```

C++ builds a cathedral around the greeting, then makes the cathedral portable. Templates hum in the rafters. Objects introduce themselves to other objects and pretend this has always been normal. Still, through all that engineered weather, the message gets out: a small human sound carried by gears, mirrors, inheritance, and a very polite stream. Somewhere in C++, a simple sentence is inheriting properties from a sentence that inherited properties from an idea somebody had in 1985. The family reunion is enormous and everyone has brought a header. `std::cout` is a public fountain in a city of namespaces. The greeting travels through its pipes, leaves the marble mouth, and lands in the square with an impeccable splash. This language dreams in blueprints. Before it sees a bird, it wants to know the bird's interface, its lifetime, its constructor, and whether it can be moved without upsetting the sky. There is grandeur in that impulse. A cathedral needs arches; an operating system needs discipline; an orbital satellite cannot rely on a shrug and good intentions. The angle brackets are mountain passes. The double colons are secret passages between fortified districts. Somewhere inside, the world is being escorted by a courteous template. C++ does not forget the cost of power. It keeps the receipt folded in its wallet, next to a photograph of the bare metal it once knew. And when the final newline comes, the cathedral windows briefly fill with daylight. For a second, complexity is not a burden but a vast instrument tuned to one human syllable.

13. Chapel

```
proc main() {
  writeln("Hello, World!");
}
```

Chapel builds its hello with a view toward distances. It imagines computation spread across rooms, racks, cities, and still wants the sentence to arrive before the coffee cools. The language looks at a crowd of processors and sees not a mob but a choir awaiting a conductor's smallest gesture.

14. Clojure

```
(println "Hello, World!")
```

Clojure keeps a small fire in the old Lisp forest and invites the modern world to warm its hands. Immutable things gather around it, unchanged yet companionable. The greeting becomes a list, and the list becomes an action without forgetting where it came from. This is memory made architectural. One phrase exits the parentheses and joins the stream. It carries no baggage except its own bright, declarative existence.

15. COBOL

```
       IDENTIFICATION DIVISION.
       PROGRAM-ID. HELLO-WORLD.
       PROCEDURE DIVISION.
           DISPLAY "HELLO, WORLD!".
           STOP RUN.
```

COBOL writes the greeting in an office with high ceilings and dependable filing cabinets. The sentence is formal because it expects to be found again thirty years from now. There is dignity in a program that introduces itself before beginning. It believes that names matter, that procedures deserve titles, that the ledger should balance even at dawn.



Small patterns, wide horizons.

16. Common Lisp

```
(format t "Hello, World!~%")
```

Common Lisp receives the phrase inside a pair of parentheses, as though language itself were taking a deep breath around it. The oldest ideas in the room nod approvingly. The symbols are not decorations; they are handles on reality. Pick one up carefully and it may reveal a new way for thought to fold back and inspect itself. The greeting appears, ancient and immediate. It feels less like a command than a spell recovered from a library beneath the sea.

17. Crystal

```
puts "Hello, World!"
```

Crystal lets the phrase form with the clarity of a bell grown inside a compiler. It keeps Ruby's warmth, then gives the sound a harder, brighter edge. The greeting becomes a small mineral held to the light. You can see through it, but it does not shatter when the world asks it to run fast. Elegance is sometimes a shortcut through a forest. Crystal takes one, leaving a clean trail of sparks behind it.

18. D

```
import std.stdio;

void main() {
        writeln("Hello, World!");
}
```

D speaks with one foot in C's machine room and another in a newer city of stronger guarantees. The greeting leaves a modern window, but somewhere underneath, the old metal staircase still knows every step by name. It is a language built for the useful tension between raw power and the desire not to lose one's way in it.

19. Dart

```
void main() {
  print('Hello, World!');
}
```

Dart sends its hello toward interfaces that have not yet been touched. It can already hear fingers approaching glass, carrying intentions too quick to name. The function is a compact stage, cleanly lit. A message appears, ready to become part of an application that wakes in somebody's palm. The world answers from a pocket, a tablet, a screen at the end of a tired day. The greeting has learned to travel lightly.

20. Delphi

```
program HelloWorld;

begin
    Writeln('Hello, World!');
end.
```

Delphi gives the phrase a neat desk, a clear beginning, and a decisive final period, as if every program were a letter mailed from a carefully ordered country. The world appears on cue, briefly illuminated by a language that believes elegance can wear practical shoes.

```
                    .       *       .
                .       /\_/\       .
            *       ( o.o )       *
                .       > ^ <       .
                    .       *       .
```

A shape becomes a path.

21. Elixir

```
IO.puts("Hello, World!")
```

Elixir sends the greeting through a city of small, resilient voices. One process speaks; another may listen; neither need carry the whole sky alone. There is a phoenix somewhere in its imagination, not as a symbol of drama, but as a practical design principle: begin again, keep

the message moving. `IO.puts` lifts the phrase into daylight. Even the console feels briefly communal, a plaza where concurrent dreams exchange directions.

22. Elm

```
module Main exposing (main)

import Html exposing (text)

main =
        text "Hello, World!"
```

Elm does not merely print the world; it places the words where a browser can hold them without dropping the state of the universe. The greeting becomes a small, reliable piece of architecture, bright enough for a screen and careful enough for a changing afternoon.

23. Erlang

```
-module(hello).
-export([main/0]).

main() ->
    io:format("Hello, World!~n").
```

Erlang distributes its attention like lanterns across a dark field. A greeting is never entirely alone when the system has learned how to survive distance. The period at the end is decisive, a small stone placed on the page. The message has completed its journey and knows where its boundary lies. Far away, another process could receive the same hello and continue it. The world is not one room but a network of rooms, each with a light beneath the door.

24. Euphoria

```
puts(1, "Hello, World!\n")
```

Euphoria directs the sentence toward a numbered stream as though the machine were a theatre with several doors and each door had a different kind of echo. Its name promises

delight, and the plain little output feels like the promise keeping itself. The message leaves on channel one and returns to us as a bright, uncomplicated fact.

25. F#

```
printfn "Hello, World!"
```

F# writes the sentence as though it were the first line of a proof that also happens to enjoy jazz. The result has crisp edges and a pulse beneath them. Functions pass through the landscape like trains that never lose their destination. The greeting boards one briefly, waves from the window, and arrives exactly where expected.



A pause between pulses.

26. Factor

```
"Hello, World!" print
```

Factor lets the words arrive before the verb, like a gift placed on a table before anyone explains the occasion. The stack waits underneath, patient as a well. Meaning accumulates in quiet layers. One thing meets another; a transformation happens; the room somehow becomes more orderly than before. The greeting rises from this hidden depth with an odd grace, a pearl brought up by a language that enjoys diving.

27. Fantom

```
class HelloWorld {
    static Void main() {
        echo("Hello, World!")
    }
}
```

Fantom walks through several runtimes without troubling itself to announce which floor it has reached, a traveler carrying one compact suitcase through many climates. The hello it echoes is practical and faintly spectral, a voice that knows how to inhabit more than one kind of machine. It passes through the wall between platforms and remains recognizably itself.

28. Forth

```
: hello  ." Hello, World!" ;
hello
```

Forth is a pocketknife of a language: small, sharp, and strangely capable of building a house if given enough time. It makes new words from old ones without any fuss. There is something prehistoric and futuristic about this. A few marks on a surface, and suddenly the surface has learned a new trick.

29. Fortran

```
program hello_world
    print *, "Hello, World!"
end program hello_world
```

Fortran speaks from the deep archive of computation, where weather, bridges, stars, and turbines have long been persuaded into numbers. Its greeting has old, trustworthy hands. After all the new languages and bright inventions, the oldest rooms are still lit. `Hello, World!` walks through one of them and hears history answer from the walls.

30. Futhark

```
entry main : []u8 = "Hello, World!\n"
```

Futhark offers the greeting as an array of luminous little units, each character a fish in a fast-moving computational river. It is a language built to think in parallel, so even this small sentence seems to stand at the edge of a much larger current.

```
                    .       *       .
                .       /\_/\       .
            *       ( o.o )       *
                .       > ^ <       .
                    .       *       .
```

Lines turning toward light.

31. Go

```
package main

import "fmt"

func main() {
    fmt.Println("Hello, World!")
}
```

Go packs lightly. It brings a function, a package, one honest import, and the kind of silence that appears just before a useful thing is built. Its hello does not echo in a marble hall. It crosses a workbench, past bright tools and coiled wire, and lands in the air ready to become part of something larger. Go has the posture of a person who owns one excellent jacket and knows how to repair a bicycle. It does not confuse plainness with a lack of depth. The package declaration is a town sign at the edge of a map: you are here. From this modest coordinate, whole systems may grow outward like roots beneath a factory floor. `fmt` is a little box of practical instruments. It does not promise enlightenment. It hands you a hammer that fits the nail and watches you build something that survives rain. There is poetry in refusing unnecessary ornament. A spare room can hold a good conversation. A small function can carry enough electricity to wake a continent. Go trusts the reader's hands. It wants code to look like code, not like an encrypted confession from a monastery whose monks all loved punctuation. The greeting passes through the program with clean edges. It does not linger in a hallway discussing metaphysics, though the hallway itself is quietly full of metaphysics. Concurrency waits nearby like a river with several currents. Even before we use it, we can hear water moving behind the walls, taking messages to places we have not built yet. And so `Hello, World!` is delivered with the satisfaction of a tool returning to its drawer. The universe remains mysterious, but the workbench is orderly and the lamp is on.

32. Groovy

```
println 'Hello, World!'
```

Groovy says hello with its collar open and its syntax loosened just enough to let air through. The phrase arrives as if a serious platform briefly remembered that software can dance without falling over.

33. Hack

```
<?hh
<<__EntryPoint>>
function main(): void {
    echo "Hello, World!\n";
}
```

Hack takes a familiar web-shaped voice and gives it a stricter map, drawing brighter boundaries around the parts that once wandered freely. Its greeting crosses those boundaries with a badge, a type, and the assurance that even a lively city benefits from good street signs. The words reach the page without having to pretend that safety is a joyless idea.

34. Haskell

```
main :: IO ()
main = putStrLn "Hello, World!"
```

Haskell does not rush toward the world. It describes the greeting with such care that the greeting seems to exist first in a purer country, untouched by the console. Then `IO` opens one narrow gate, and reality is allowed in. The words pass through, immaculate, wearing the quiet expression of a theorem that has learned to dance. Laziness waits nearby in a hammock made of parentheses. Nothing moves until it is needed, and when it moves, it knows exactly why.

35. Haxe

```
class Main {
    static function main() {
        trace("Hello, World!");
    }
}
```

Haxe sends the same sentence toward many possible destinations, letting the target decide what sort of sky it will wake beneath. The trace is a small beacon across platforms, proof that a thought may keep its shape while changing worlds.



A measured drift.

36. HolyC

```
U0 Main() {
        Print("Hello, World!\n");
}
```

HolyC speaks from a singular imagined kingdom, where the operating system and its language share the same vivid private weather. Its hello feels like a message painted directly onto the wall of the machine, strange, earnest, and impossible to mistake for ordinary wallpaper. The console answers from a place that has chosen to make its own rules visible.

37. Icon

```
procedure main()
    write("Hello, World!")
end
```

Icon regards the greeting as a possible success, which makes it more interesting than a certainty. The language has always known that a search can be part of the answer. The

words appear at the end of a small procedure, as if they have completed a journey through a forest of alternatives. Choice flickers in the background like a distant sign. The hello is one path selected, warm and real beneath our feet.

38. Idris

```
module Main

main : IO ()
main = putStrLn "Hello, World!"
```

Idris lets the greeting carry a type like a passport stamped by logic itself, then sends it through the narrow border of `IO`. The result is a sentence with a remarkably clean conscience, as though it has explained not only what it says but why it is permitted to say it.

39. Inform 7

```
"Hello World" by An Author

When play begins:
        say "Hello, World!".
```

Inform 7 makes the greeting happen when play begins, because in its world code is already halfway to fiction and every sentence knows it might become a room. The words do not appear in a console so much as in the first breath of an adventure that has just opened its eyes.

40. Io

```
"Hello, World!" println
```

Io lets the string take the initiative, handing itself the instruction to speak as if objects were conversational beings with excellent instincts. The greeting climbs out of the quotation marks, turns toward us, and demonstrates how little ceremony a message truly needs.

```
                    .       *       .
                .       /\_/\       .
            *       ( o.o )       *
                .       > ^ <       .
                    .       *       .
```

Hidden order, briefly visible.

41. J

```
smoutput 'Hello, World!'
```

J compresses thought until it acquires the density of a small star, then gives the greeting to an output verb and lets the symbols do their quiet work. The result is spare but not empty, a tiny equation whose answer happens to be a human welcome.

42. Java

```
class HelloWorld {
    public static void main(String[] args) {
        System.out.println("Hello, World!");
    }
}
```

Java begins with a house called a class, because even a greeting deserves an address. Before it can speak, it locates the front door, checks the rules, and invites the entire morning inside. The class is both name and container, a small apartment for an idea. `HelloWorld` hangs above the door in brass letters, though the resident is only a sentence with excellent manners. `public static void main` sounds like a legal oath recited by a choir of robots. Beneath the ceremony, it means something almost tender: begin here; let everyone find the beginning. There is comfort in repetition. The greeting crosses operating systems as a traveler crosses borders with the same photograph tucked inside a passport. Java understands that a system can be very large and still need a front door. It places the handle exactly where the next person expects it, which is an underrated kind of mercy. The semicolon closes the phrase like a clasp on an old book. The message waits there, balanced between discipline and warmth, ready to become an artifact of execution. Then the runtime

exhales, and the words arrive. In the vast portable house, somebody has lit one window, and the world can see it from the road.

43. JavaScript

```
console.log("Hello, World!");
```

JavaScript speaks from a room with no walls. It calls into the console, and the console becomes a puddle, and the puddle reflects a thousand tabs arguing softly about the weather. Here, reality is event-shaped. The greeting does not simply happen; it arrives when the cosmos has finished loading, wearing a name tag and carrying a tiny bell. The semicolon is a period after an argument with the future. It says: this thought may stop here, though nothing in the browser is ever truly finished. JavaScript was born near the marketplace, where signs flicker and people change their minds mid-sentence. It knows the sacred panic of a button waiting to be pressed by a stranger with coffee on their sleeve. The web is a city made of doors pretending to be windows. This line is taped to one of them, a handwritten note from the inside: yes, someone is awake in here. Every console is a private basement beneath a public building. Down there, developers ask the walls difficult questions and receive answers that are occasionally shaped like warnings. Time in JavaScript has loose shoelaces. A promise walks past, carrying tomorrow in a paper bag. A callback waves from a train that has already departed and somehow arrives first. There is a strange democracy in the runtime. One tiny script may speak beside enormous systems, beside games, banks, poems, menus, lunar calendars, and a photograph of somebody's lunch. The phrase survives all of this with its coat unbuttoned. Hello, World: a street musician playing one clear note beneath the overpass of an infinite afternoon.

44. Julia

```
println("Hello, World!")
```

Julia greets the world with a notebook open on its knees and a rocket diagram in the margin. It knows that speed and thoughtfulness need not be enemies at dinner. The words arrive with numerical ambition. Somewhere beneath them, arrays are dreaming of galaxies and functions are making room for impossible precision. This is a language that wants the experiment to remain alive while it is being measured. The hello is a spark held up to a very large engine.

45. Kotlin

```
fun main() {
    println("Hello, World!")
}
```

Kotlin steps into the room with the confidence of a bridge that knows both riverbanks. It speaks plainly, but carries a modern little constellation of guarantees in its pocket. The greeting rises from `main` without ceremony. Null is asked to wait outside, where it may consider the consequences of arriving unannounced. Here the world is bright, concise, and unexpectedly well-appointed. A single line crosses the threshold, leaving behind the feeling that careful design can still have a sense of humor.

```
                    .       *       .
                .       /\_/\       .
            *       ( o.o )       *
                .       > ^ <       .
                    .       *       .
```

A window made of symbols.

46. Lingo

```
on exitFrame
    put "Hello, World!"
end
```

Lingo places the message at the edge of a frame, where animation and code once met in the brightly lit hallways of interactive media. The world is spoken just as the scene changes, a note slipped between one moving image and the next.

47. LiveCode

```
put "Hello, World!" into message box
```

LiveCode prefers a sentence that resembles an instruction given across a desk, direct enough that the machine feels almost like a helpful colleague. The message box receives the

greeting with the uncomplicated hospitality of an open window. Nothing has to be hidden behind syntax when the thought already knows how it wants to be said.

48. Logo

```
print [Hello, World!]
```

Logo sends the phrase out with the same gentle clarity that once taught small cursors to draw impossible shapes. Somewhere a turtle pauses, considers the world, and carries the greeting forward one careful step at a time.

49. Lua

```
print("Hello, World!")
```

Lua is a small moon with a surprising gravitational pull. It slips into games, tools, and hidden places where a little language must learn to be nimble. Its print statement is almost a whisper, but whispers travel far in a quiet engine. The phrase crosses a landscape of sprites and systems, lit from within. There is no need to announce one's size to alter a world. A tiny script can move a door, summon rain, or teach a blank screen how to say hello.

50. MATLAB

```
disp('Hello, World!')
```

MATLAB sees the phrase as a vector waiting to become a wave. Its hello has the faint hum of laboratories, chalk dust, and graphs unfolding like mechanical flowers. `disp` places the words where an instrument can see them. The room fills with coordinate axes, each one promising that confusion can be rotated into clarity. Somewhere beyond the screen, an equation adjusts its spectacles. The greeting is not an answer yet, but it is a stable initial condition.



The rhythm gathers.

Foretext: The Fifty-First Room

At the midpoint, the book develops a second entrance. This is not a mistake, although mistakes are often doors wearing badly printed signs. The first fifty languages have already said hello, and the reader may have begun to believe the ritual is understood. There is a heading, a code block, a brief electricity, a return to prose. The pattern has become familiar enough to disappear. Familiarity is a powerful solvent; it dissolves astonishment first. So this room has been placed here to interrupt the solvent. It asks you to look up from the procession and remember that you are standing inside an arrangement of repeated beginnings.

The fifty-first voice does not know it is fifty-first. It has no awareness of the entries behind it, no sense that it has arrived late, no resentment toward the languages that came before. A program does not stand in a queue unless someone teaches it to imagine the queue. Yet you know the count, and the count changes the atmosphere. Fifty greetings have accumulated in the reader's mind like lights seen from a distant train. Each one is small. Together they imply a city. You cannot visit every room in that city, but you can see that the windows are not random. Someone has been awake in all of them. Someone has asked a machine to speak and waited to see whether the machine would agree.

There is an odd consequence to repetition: the original phrase begins to detach from ordinary language. Read it often enough and `Hello, World!` stops behaving like a greeting. It becomes a shape, then a sound, then a small machine of its own. The comma becomes a hinge. The exclamation mark becomes a nail catching light. The word world becomes less a planet than a container for whatever can be addressed. The word hello becomes less an act of welcome than a test of whether anything is listening. By now, the phrase may have become slightly unfamiliar to you. Good. Unfamiliarity is not always a loss of meaning; sometimes it is meaning returning from a long walk with a different face.

This is the point at which the book begins to read the reader. Not literally, not with the clumsy certainty of a fortune teller, but through the ordinary mechanism of attention. You

supplied the pauses between sections. You gave weight to one language and passed quickly over another. You recognized some fragments as home and others as weather from a country you did not know existed. You may have felt a small pride at recognition, or a small annoyance at opacity, or the rare clean pleasure of finding something that refuses to become useful immediately. None of these responses is outside the book. They are part of its circuitry. A text is not a container; it is a meeting. It changes shape according to the mind that enters it, and the mind leaves with an extra room added somewhere behind its usual furniture.

The next entries are not a second half in the ordinary sense. They are the underside of the first half revealed by turning the page over. The same letters may appear, but their shadows have moved. Languages close to the metal will speak beside languages made for proof, play, money, hardware, documents, simulations, and strange old operating systems with private calendars. Some will look familiar enough to invite assumptions. Others will arrive with punctuation that behaves like architecture from a dream. Give each one a moment before deciding it is obscure. Obscurity is often just usefulness observed from the wrong distance. A language can be dormant and still have altered the riverbed. A language can have few speakers and still contain a thought no popular tool ever needed to express.

Imagine, for a moment, that all ninety-nine greetings are occurring at once. Not played one after another in a terminal, but spoken simultaneously from every kind of machine that has made room for them. The sound would not be a chorus exactly. It would be too uneven, too full of formats and encodings and runtimes, too crowded with different assumptions about what it means to arrive. It might resemble static. It might resemble an orchestra warming up in a building with no roof. It might resemble silence after you realize silence has been listening. Yet inside that impossible overlap, the phrase would remain intact. A small human arrangement of words would cross every difference and make itself known without becoming identical to itself.

There is no instruction hidden in this foretext, despite its position among instructions. You do not need to memorize the entries ahead. You do not need to choose a favorite, defend a hierarchy, or convert the anthology into a productivity plan. The useful act is stranger and simpler: continue. Continue with the mild suspicion that every technical artifact has an inner weather. Continue knowing that a formal system can be exact without becoming dead. Continue as though the next code block might be a window rather than a wall. The reader who continues is not the same reader who began, even when the difference is too small to name.

When you return to the languages below, let them be new again. Let the fifty-first voice feel less like an item in a list and more like a corridor opening where a corridor should not fit. The book has not changed its subject. It is still saying hello. But the greeting has crossed its own midpoint, and that is enough to make the room tilt slightly. In the tilt, a new question appears: if the world can be greeted in ninety-nine ways, how many ways has it already greeted you without waiting for an answer?

51. Mercury

```
:- module hello.
:- interface.
:- import_module io.
:- pred main(io::di, io::uo) is det.
:- implementation.

main(!IO) :-
        io.write_string("Hello, World!\n", !IO).
```

Mercury holds the world's state in both hands and marks how it passes from one moment to the next, as though time itself deserved an explicit argument. The greeting moves through that carefully tracked current and arrives with nothing hidden under its coat.

52. Modula-2

```
MODULE Hello;
FROM InOut IMPORT WriteString, WriteLn;
BEGIN
    WriteString("Hello, World!");
    WriteLn
END Hello.
```

Modula-2 gives the greeting a module to live in, a small well-built district with clear imports and no unnecessary alleys. The message walks through its designated gate, then settles into the console with the satisfaction of a well-planned journey.

53. Mojo

```
fn main():
        print("Hello, World!")
```

Mojo carries the hello from a familiar, readable surface toward the fast interior of modern computation, where accelerators wait like engines behind velvet curtains. The sentence remains plain, but the road beneath it is already humming with unreasonable possibility.

54. MUMPS

```
HELLO W "Hello, World!",! Q
```

MUMPS writes in a compact clinical shorthand that has spent decades near the beating heart of records and systems that cannot casually forget. Its hello is terse, but it has the gravity of a note entered into a chart that will matter tomorrow.

55. Nim

```
echo "Hello, World!"
```

Nim gives the phrase a clean, nimble body. It looks as though it could fit in a poem, then surprises you by also knowing how to carry a heavy machine. The echo crosses the console and comes back with no unnecessary ceremony. Somewhere, the language is quietly rearranging itself into whatever shape the task needs. This adaptability is not shapelessness. It is the skill of a river finding the right curve in stone.



A thought in transit.

56. Oberon

```
MODULE Hello;
IMPORT Out;
BEGIN
    Out.String("Hello, World!"); Out.Ln
END Hello.
```

Oberon keeps the language light enough that each part can be held up to the sun and inspected. Its greeting is an old clean shape, a message from a school of design that trusted restraint to make room for thought.

57. Objective-C

```
#import <Foundation/Foundation.h>

int main(void) {
    @autoreleasepool {
        NSLog(@"Hello, World!");
    }
    return 0;
}
```

Objective-C sends messages as though every object were a small, opinionated person with a telephone. The greeting rings through a city of brackets and receives a courteous answer. There is history in its accent: C's sturdy bones, Smalltalk's conversational spirit, the polished glass of many applications held in a hand. The world is logged, and somewhere an autorelease pool carries away the cups after a very precise party.

58. OCaml

```
print_endline "Hello, World!";;
```

The double semicolon is a tiny closing gate. Behind it, values walk in orderly procession, each carrying a truth it can defend. The greeting reaches daylight cleanly. It has been considered, inferred, and released without ever losing its strange little soul.

59. Odin

```
package main

import "core:fmt"

main :: proc() {
        fmt.println("Hello, World!")
}
```

Odin meets the machine with a clear gaze, unwilling to hide the tools behind ornamental curtains. The greeting comes through a practical core library, solid as a peg driven into a workbench before the larger structure rises. It is a short line with the weight and grain of something built to be understood under bright light.

60. Oz

```
{Browse 'Hello, World!'}
```

Oz opens a browser into a different kind of computation, one where values, threads, and constraints may share a table without losing their manners. The word `Browse` feels exactly right: the world is not just output, but an object of curious attention.

```
                    .       *       .
                .       /\_/\       .
            *       ( o.o )       *
                .       > ^ <       .
                    .       *       .
```

Contours of a common phrase.

61. Pascal

```
program HelloWorld;
begin
    Writeln('Hello, World!');
end.
```

Pascal greets the world between a named beginning and a final period, with all the composure of a formal letter written in excellent ink. Its clarity has taught generations of programs how to stand upright before they learn to run.

62. Perl

```
print "Hello, World!\n";
```

Perl finds the greeting in the sleeve of a wizard's coat, beside a regular expression and three useful tools whose names have been forgotten by everyone except the machine. It believes there is more than one road through the forest, and it has walked several at once. The destination is still the same bright phrase. The punctuation makes a small carnival of itself. Yet through the confetti, the words land precisely where they were always meant to be.

63. PHP

```
<?php
echo "Hello, World!\n";
```

PHP lives close to the surface, where pages breathe and forms remember what hands have asked of them. It speaks through the web's long afternoon with sleeves rolled up. The opening tag is a curtain lifted in a small theatre. Behind it, the server has been waiting in the dark, rehearsing a sentence it has said a billion times. `echo` does not philosophize about expression. It sends the message out, and somehow that practical insistence becomes its own kind of poetry.

64. Pike

```
int main() {
    write("Hello, World!\n");
    return 0;
}
```

Pike sends the greeting through a language that knows C's terrain but has built a few more bridges across it. The output is direct, sturdy, and pleasantly unconcerned with whether anybody has heard of the path it took.

65. Pony

```
actor Main
    new create(env: Env) =>
        env.out.print("Hello, World!")
```

Pony lets an actor carry the greeting into the world, making communication feel less like a shared room and more like a well-timed exchange across a stage. The sentence arrives without a fight over who owns the furniture, which is a small miracle worth preserving.



A bright thread through noise.

66. PowerShell

```
Write-Output "Hello, World!"
```

PowerShell sees not just text but objects moving through a system like well-dressed planets. The greeting enters the pipeline with its identity intact. To write output is to release an object from one room into another, where it may be inspected, transformed, or simply admired for knowing how to be itself. The words arrive on Windows with the calm authority of a clerk who has found exactly the form the universe requires.

67. Prolog

```
:- initialization(main).

main :-
        writeln('Hello, World!').
```

Prolog declares the destination and lets the logic find its way there, as if the greeting were the answer to a question we forgot to ask. The final period clicks into place, and the world

becomes a fact for one perfectly reasonable moment.

68. PureScript

```
module Main where

import Effect.Console (log)

main = log "Hello, World!"
```

PureScript sends the phrase through an effect instead of smuggling it past the type system, treating reality as something significant enough to name. The result is not less poetic for being explicit; it is a lantern carefully carried through a dark type signature.

69. Python

```
print("Hello, World!")
```

Python does not knock. It opens its palms and lets the sentence fly out, bright as a paper lantern escaping a festival. Somewhere beneath the friendly syntax, a small universe agrees to be named. The world hears itself called and, for one immaculate instant, remembers it was waiting. Then the interpreter folds the moment into its sleeve and asks what we would like to dream next. There is a kindness in `print`: no scaffolding, no brass band, merely a request made aloud. The language trusts that a thought can begin in daylight without having to explain its credentials. Imagine the first programmer as a person standing on the shore of an ocean that is also a mirror. They say hello, and the reflection says it too, except the reflection has learned the grammar of electric storms. Python waits behind the phrase like a quiet librarian with a secret tattoo of a serpent. It has seen every impossible question shelved beside every ordinary answer, and it never once asks why they are neighbors. The quotation marks are two small doorways facing each other. Between them, the world sits cross-legged, perfectly pronounceable, waiting for the current to notice it. There are languages that make you assemble a telescope before you may look at the moon. Python hands you the moon, warm from somebody else's pocket, and asks what you see in the silver dust. Perhaps a program is only a ritual for persuading silence to change clothes. A blank screen becomes a witness. A sentence enters. The dark does not disappear, but it learns a name for its companion. In the terminal, the greeting remains after its maker has

gone to make tea, sleep badly, fall in love, forget passwords, become an ancestor. This is the tender absurdity of a line that runs.

70. R

```
cat("Hello, World!\n")
```

R hears the greeting as a point in a vast cloud of possible points. It wonders whether the world has a mean, a median, an outlier hiding under a velvet curtain. The sentence lands on the console like a clean observation. One row, one variable, one uncomplicated fact in a universe that prefers messy data.

```
                    .       *       .
                .       /\_/\       .
            *       ( o.o )       *
                .       > ^ <       .
                    .       *       .
```

A map without borders.

71. Racket

```
#lang racket
(displayln "Hello, World!")
```

Racket begins by declaring its world, then lets the greeting unfold inside it. Language is not a fixed building here; it is a workshop where the walls can be moved. The parentheses form a nest around the phrase. Within that nest, new dialects are already hatching, each with its own way to ask what code might be. The message leaves the display and finds us waiting. Even the most experimental syntax needs an audience for its first clear note.

72. Raku

```
say "Hello, World!";
```

Raku says hello as though language were a generous bazaar full of strange tools, bright fabrics, and several valid routes to the same door. The `say` is a human-sized verb, and that

alone makes the greeting feel less like an order than an invitation.

73. REBOL

```
print "Hello, World!"
```

REBOL speaks in blocks and bindings, seeing the world as a sequence that can be rearranged into unexpectedly useful meanings. Its hello feels like a message delivered by a small vessel crossing an ocean of protocols.

74. Red

```
print "Hello, World!"
```

Red places the phrase on the console with a color in its name and portability in its stride. The greeting is compact, but it carries the faint charge of a tool designed to travel where larger languages would need a convoy.

75. Ring

```
see "Hello, World!" + nl
```

Ring lets the greeting appear with a word as simple as `see`, as though the crucial act were not producing the world but noticing it. The newline follows like a soft footstep, making space for whatever the next moment wants to become.



Many paths, one beginning.

76. RPG

```
**free
dsply 'Hello, World!';
*inlr = *on;
```

RPG brings the greeting from a world of business machines, long-lived systems, and code that learned durability before novelty was fashionable. The display is small, but it carries the steady rhythm of work that must still be correct on an ordinary Tuesday decades later. The phrase appears like a familiar indicator light on a control panel that has outlived several fashions.

77. Ruby

```
puts "Hello, World!"
```

Ruby says hello as though it has just remembered the world is beautiful and wishes to tell it before the kettle boils. There is no ceremony beyond the sentence, and that is the ceremony. Ruby treats a program as a conversation that happened to learn how to repeat itself. Its syntax does not march; it wanders through a garden, touching leaves to see which ones answer. `puts` is an open window. The words do not need to climb a ladder of ceremony. They simply lean out, call to the street, and accept the echo as sufficient proof of being alive. Objects gather here like guests at a midnight salon. They can be given new habits, new names, new reasons to exist, and nobody asks whether the invitation came too late. The language makes room for the idea that code may be read aloud. Not inspected like a gearbox, not defended like a border, but read as one reads a letter from an intelligent friend. In that reading, the greeting becomes a charm. It is not a command barked at matter; it is a small spell spoken plainly enough that matter agrees to play along. Ruby's world is not less exact because it is warm. A violin string is warm in the hand, after all, and still capable of finding the one note that makes a room stop breathing. The final line fades from the console, but the theatre remains. Somewhere behind the curtain, a new object has just invented a hat and is preparing to introduce itself to the moon.

78. Rust

```
fn main() {
    println!("Hello, World!");
}
```

Rust lets the greeting through the gate only after asking whether it will return everything it borrowed. This is not distrust. This is devotion wearing armor made of careful questions. When the sentence finally appears, it is alive, accounted for, and gloriously unable to fall through a trapdoor that was never built. Rust has looked into the abyss of undefined behavior and written it a letter with numbered demands. The abyss has not answered, but it no longer gets unsupervised access to the pantry. Ownership is a strange kind of love story. Things may be held, lent, moved, or released, but every gesture must admit its consequence under the clear moon of the compiler. The macro's exclamation mark is not shouting. It is striking a match. `println!` flares briefly, and in its light the architecture of caution becomes almost festive. There are languages that tell you to be careful. Rust asks what careful means, writes down the answer, and waits patiently until your program agrees. This can feel like being guided through a labyrinth by a guardian who refuses to let you sprint toward the walls. Eventually you notice the guardian has been saving you from rooms with no floor. The type system is not a prison guard but a watchmaker. It sits at a bench beneath a magnifying glass, ensuring each bright little gear will meet the next without grinding itself into sorrow. Then the words emerge, not fragile but carefully free. The world says hello back through a structure strong enough to let wonder lean against it.

79. SAS

```
%put Hello, World!;
```

SAS writes the greeting into a log where data has been made to confess its habits, its exceptions, and its stubborn hidden patterns. The macro statement is a brief flare above a landscape of tables, models, and questions too large for one person to hold unaided.

80. Scala

```
object HelloWorld extends App {
  println("Hello, World!")
}
```

Scala gives the greeting a structure grand enough to house its future selves. The object stands at the door, formal but not unfriendly, and lets the sentence become an event. In its mirror, functions become values and values become small decisions about the shape of tomorrow. The world is not merely printed; it is composed. The line ends, but the types keep

singing beneath it. They are a choir hidden behind the walls, turning logic into a kind of weather.



The signal finds its echo.

81. Scheme

```
(display "Hello, World!")
(newline)
```

Scheme reduces the furniture until the room becomes beautiful. A few forms, a few rules, and suddenly there is enough empty space for the universe to sit down. The display is simple because simplicity is not a lack; it is a blade sharpened until it can divide a question from its unnecessary costume. Then comes the newline, a small act of mercy. The world is greeted, and also given somewhere to rest.

82. Seed7

```
$ include "seed7_05.s7i";

const proc: main is func
    begin
        writeln("Hello, World!");
    end func;
```

Seed7 grows its greeting in a language that treats extensibility as a native season rather than a feature bolted on after winter. The words arrive from a small, unusual garden where syntax itself is allowed to keep evolving.

83. Simula

```
BEGIN
    OutText("Hello, World!");
    OutImage;
END;
```

Simula gives the hello an echo from the origin story of objects, a time when simulation taught code how to imagine a world full of separate things. The phrase appears, and behind it you can almost see the first small actors waiting for their turn to move.

84. Smalltalk

```
Transcript show: 'Hello, World!'; cr.
```

Smalltalk sends a message to a transcript, because in its universe everything is already engaged in a conversation. The greeting is not forced from the system; it is politely asked to appear, and the system responds in kind.

85. Solidity

```
pragma solidity ^0.8.0;

contract HelloWorld {
        function hello() public pure returns (string memory) {
                return "Hello, World!";
        }
}
```

Solidity seals the greeting inside a contract and lets anyone ask for it, a message made durable enough to outlast the person who deployed it. The words wait on a distributed ledger like a note carved into a public monument, patient and expensive to alter.

```
                          .       *       .
                      .      /\_/\      .
                  *       ( o.o )       *
                      .      > ^ <      .
                          .       *       .
```

A quiet constellation.

86. SPARK

```
with Ada.Text_IO; use Ada.Text_IO;

procedure Hello_World with SPARK_Mode is
begin
        Put_Line ("Hello, World!");
end Hello_World;
```

SPARK lets the greeting stand beneath a stricter light, inviting formal reasoning to inspect the path by which a simple output becomes trustworthy. It is Ada's careful relative, arriving with a proof obligation and leaving the room safer than it found it.

87. Standard ML

```
print "Hello, World!\n";
```

Standard ML sends the phrase through a language where types are inferred with a kind of quiet clairvoyance. The output is modest, but it rests on a foundation that likes to know what every value means before the world becomes complicated.

88. SuperCollider

```
"Hello, World!".postln;
```

SuperCollider posts the greeting as though it were the first audible pulse in a composition that has not yet chosen its instruments. The console becomes a rehearsal room, and the words hover there with the anticipation of a sound about to become space.

89. Swift

```
print("Hello, World!")
```

Swift releases its greeting like a polished stone into clear water. The ripples are precise, but nothing about them feels cold; the language wants speed without giving up grace.

Somewhere, an apple-shaped myth opens a door onto a screen. The words arrive dressed in sunlight, carrying the quiet assurance of something that knows where it belongs. To print is to make an interior thought briefly touch the glass. The world presses back, visible in the reflection, and the program smiles without a mouth.

90. Tcl

```
puts "Hello, World!"
```

Tcl offers the sentence with the directness of a tool designed to connect things that were never quite meant to meet. Its name once meant command language, and even this small hello feels ready to orchestrate a larger room.

```
                    .       *       .
                .       /\_/\       .
            *       ( o.o )       *
                .       > ^ <       .
                    .       *       .
```

The pattern opens outward.

91. TypeScript

```
console.log("Hello, World!");
```

TypeScript greets the world after first asking what, precisely, the world believes itself to be. Is it a string? A promise? A creature with optional properties and a complicated childhood? The answer is filed in a cabinet of blue light. Only then may the sentence leave the building, fully labeled, dignified, and slightly suspicious of infinity. To give something a type is to draw a circle around a mystery and write, gently, do not panic. The mystery remains inside, drinking tea, but at least the furniture has names now. TypeScript listens to JavaScript tell a sprawling dream, then asks which parts were literal. It is not trying to spoil the dream. It is trying to keep the staircase from wandering into the sea. The compiler is a stern astrologer with a chalkboard. It reads the arrangement of symbols and says, with unusual compassion, that this constellation cannot possibly be a number. In another universe, all values arrive with passports. They queue under a fluorescent sky, declare their intentions, and pass through the border only when the guard recognizes their shape. The greeting itself is simple

enough to evade philosophy. Yet its existence says that even a simple thing benefits from a map, especially when the map is drawn by someone who has been lost before. There is comfort in an error before the error has happened. It is like hearing the kettle whistle in a dream and waking before the house fills with steam. TypeScript does not defeat chaos; that would be vanity. It gives chaos a clipboard, a meeting room, and a series of increasingly specific questions. Then the familiar words are released into the console, carrying both their message and their paperwork. The world receives them, stamped and shining, at the border of the possible.

92. V

```
fn main() {
    println('Hello, World!')
}
```

V approaches the greeting with an uncluttered desk and a sharp pencil. It wants the obvious thing to remain obvious, a radical wish in a world fond of decorative fog. The function opens, the phrase appears, and the function closes. Between those two braces, the universe is briefly very easy to inspect. Clarity is not the end of mystery. It is the window through which mystery becomes visible without tripping over the furniture.

93. Vala

```
void main() {
        print("Hello, World!\n");
}
```

Vala lets the greeting borrow a familiar C-shaped outline while drawing on a richer object world beneath it. The result is a sentence that feels both native to the metal and at ease among higher-level ideas.

94. Verilog

```
module hello_world;
  initial begin
    $display("Hello, World!");
  end
endmodule
```

Verilog lets the greeting occur in a world where time can be designed. Signals wait behind the curtain, ready to become edges, pulses, and decisions made in silicon. The display is only a simulation, perhaps, but simulations are dreams that engineers have taught to keep appointments. Somewhere a circuit is learning the shape of a future it will one day enact at the speed of light.

95. VHDL

```
entity hello_world is
end hello_world;

architecture test of hello_world is
begin
    process
    begin
        report "Hello, World!" severity note;
        wait;
    end process;
end test;
```

VHDL lets the greeting occur in the measured theatre of a simulated circuit, where time advances in deliberate increments and signals keep their appointments. The report is a note from the future hardware, a voice before the silicon has fully learned to speak. In the pause after the message, a clock waits patiently for the next edge of the world.

```
                    .       *       .
                .       /\_/\       .
            *       ( o.o )       *
                .       > ^ <       .
                    .       *       .
```

Ninety-nine paths meet here.

96. Visual Basic .NET

```
Module HelloWorld
    Sub Main()
        Console.WriteLine("Hello, World!")
    End Sub
End Module
```

Visual Basic .NET welcomes the world with the full warmth of words that want to be read by people. Its structure says what it means and means what it says. The module is a tidy room. The subroutine opens the curtains. The greeting takes its place in the window without pretending the sunlight belongs to it. There is a sturdy friendliness here, a belief that power need not make itself obscure in order to feel important.

97. Wolfram Language

```
Print["Hello, World!"]
```

The Wolfram Language sees a greeting as a symbolic object with an afterlife. It could be transformed, plotted, solved for, held up to the universe and asked what it resembles. For one moment, hello is not merely an output. It is an expression waiting to discover what else it can become.

98. XQuery

```
"Hello, World!"
```

XQuery returns the greeting as a value drawn from a world of documents, paths, and carefully selected fragments of meaning. At the last new threshold, the phrase does not shout into a terminal; it simply becomes the answer, cleanly lifted from the tree.

99. Zig

```
const std = @import("std");

pub fn main() !void {
    const stdout = std.io.getStdOut().writer();
    try stdout.print("Hello, World!\n", .{});
}
```

Zig gives the greeting a small box of exact tools and asks it to build without pretending the workshop has disappeared. It likes to see the joints, the grain, the honest effort of the hand. The `try` is a raised lantern over the path. Failure may exist, the language says, but it should not be allowed to hide in the grass. At the far edge of the anthology, the final hello is not an ending. It is a bright nail driven into the dark, from which another hundred languages might hang.

Ninety-nine voices. One greeting. The anthology continues.